

Engineering Architecture Report: test_ecommerce_diagram

Original File: test_ecommerce_diagram.png | Analyzed At: 2026-07-02 11:06:34

1. Architecture Summary

This architecture represents a modern, resilient e-commerce microservices platform. The system is designed to handle high transaction volumes and read-heavy operations, utilizing an API Gateway to routing traffic to decoupled backend services. Real-time asynchronous communications are managed through RabbitMQ, enabling horizontal scalability of payment processing and inventory updates.

2. Workflow Explanation

- User Browsing & Cart:** The user interacts with the React Single Page Application (SPA). Product detail reads are served via the Product Catalog Service with Redis caching.
- Checkout Initiation:** The SPA calls the Cart & Order Service to create a pending order. Order status transitions to 'unpaid'.
- Payment Processing:** The client makes a secure tokenized call to the Stripe API. Upon successful payment, Stripe calls our webhook, which updates the Order Service via RabbitMQ messages. This ensures eventual consistency and prevents database locks during high traffic peaks.
- Inventory & Notification:** Once paid, the Inventory Service reduces stock counts and the Notification Service dispatches a transactional confirmation email to the user.

3. Technology Stack

Frontend: React.js, Tailwind CSS, Vite, Redux Toolkit

Backend: Python, FastAPI, RabbitMQ (Message Broker), Stripe SDK

Database: PostgreSQL (Primary DB), Redis (Session & Catalog Caching)

Devops: Docker, AWS ECS (Fargate), Nginx API Gateway, GitHub Actions CI/CD

4. System Components

Name	Type	Description
React Client App	Frontend SPA	Highly responsive shopping portal, catalog, and checkout flow.
Nginx API Gateway	Reverse Proxy	SSL termination, rate limiting, and request routing to microservices.
Auth Service	Microservice	Issues JWT tokens, manages user accounts, and stores password hashes in postgres.
Order Service	Microservice	Manages order state machines, cart items, and links with payments.
Product Catalog Service	Microservice	Handles product details, categories, and inventory sync (cached with Redis).
RabbitMQ	Message Broker	Asynchronous event broker for order placements, notifications, and inventory updates.